In [17]:
```
from textblob import TextBlob
import nltk
nltk.download('punkt')
nltk.download('stopwords') # For removing common words
nltk.download('averaged_perceptron_tagger') # For Part-of-Speech tagging
nltk.download('wordnet') # For lemmatization
! python -m textblob.download_corpora
# Sample reviews with associated sentiment (as provided)
```

```
[nltk_data] Downloading package punkt to /Users/mathis/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     /Users/mathis/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /Users/mathis/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package wordnet to /Users/mathis/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package brown to /Users/mathis/nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     /Users/mathis/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package wordnet to /Users/mathis/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /Users/mathis/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package conll2000 to
[nltk_data]     /Users/mathis/nltk_data...
[nltk_data]   Package conll2000 is already up-to-date!
[nltk_data] Downloading package movie_reviews to
[nltk_data]     /Users/mathis/nltk_data...
[nltk_data]   Package movie_reviews is already up-to-date!
Finished.
```

In [18]:
```
import pandas as pd
import re
from collections import Counter

import matplotlib.pyplot as plt
import seaborn as sns
import nltk
from textblob import TextBlob, Word
from wordcloud import WordCloud
from nltk.corpus import stopwords

df = pd.read_csv("../datasets/amazon.csv")
df
```

Out[18]:

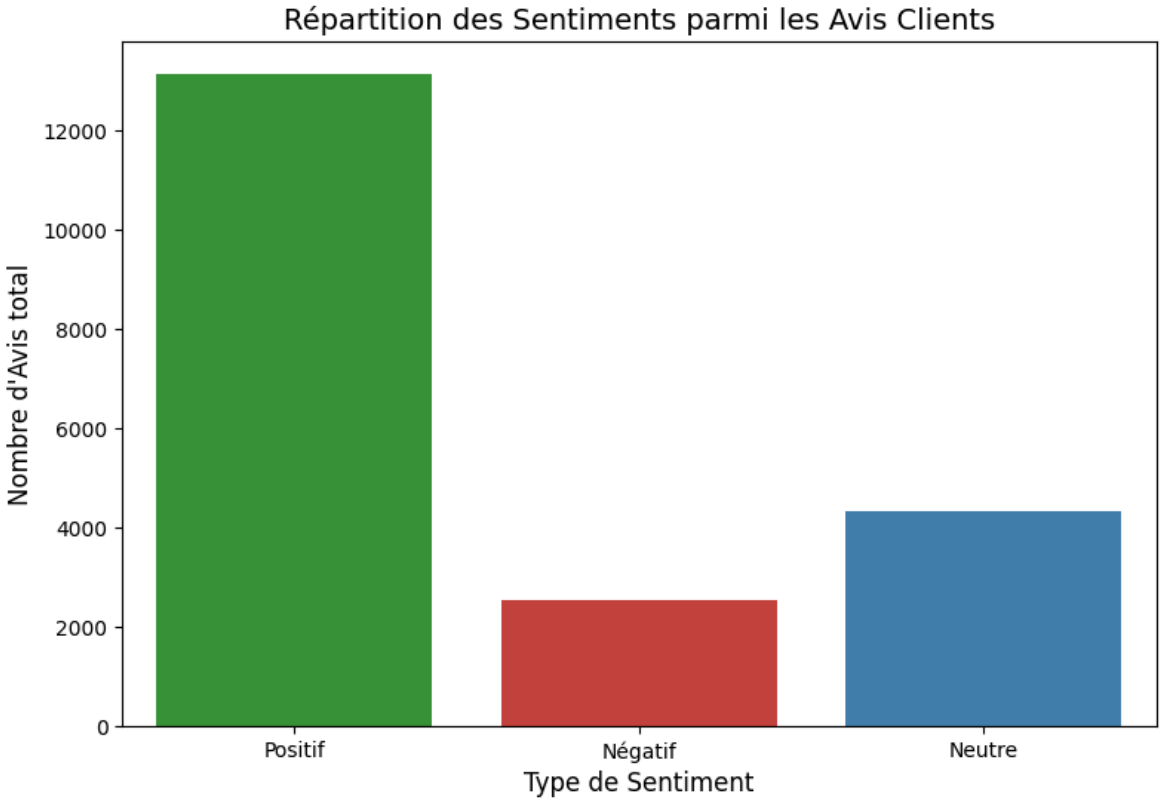
	reviewText	Positive
0	This is a one of the best apps acording to a b...	1
1	This is a pretty good version of the game for ...	1
2	this is a really cool game. there are a bunch ...	1
3	This is a silly game and can be frustrating, b...	1
4	This is a terrific game on any pad. Hrs of fun...	1
...	...	...
19995	this app is fricken stupid.it froze on the kin...	0
19996	Please add me!!!!! I need neighbors! Ginger101...	1
19997	love it! this game. is awesome. wish it had m...	1
19998	I love love love this app on my side of fashio...	1
19999	This game is a rip off. Here is a list of thin...	0

20000 rows × 2 columns

1. Can you perform a sentiment analysis on the collection of product reviews, classify each one as 'positive', 'negative', or 'neutral', and then generate a bar chart to visualize the distribution of these sentiments?

In [19]:
```
# Détermination de la bonne colonne pour les textes (on prend reviewText
colonne_texte = 'reviewText' if 'reviewText' in df.columns else df.select
liste_textes = df[colonne_texte].fillna('').astype(str)

# On filtre les commentaires vides
liste_textes = liste_textes[liste_textes.str.strip() != '']

# Calcul de la polarité pour deviner le sentiment (Entre -1 et +1)
score_polarite = liste_textes.apply(lambda avis: TextBlob(avis).sentiment

# Classification customisée en 3 catégories basée sur le score
categories_sentiment = score_polarite.apply(lambda score: 'Positif' if sc

# Comptage pour faire un joli graphe
compteur_sentiments = Counter(categories_sentiment)

# Création du barplot des sentiments
plt.figure(figsize=(9, 6))

noms_categories = list(compteur_sentiments.keys())
valeurs_categories = list(compteur_sentiments.values())

sns.barplot(x=noms_categories, y=valeurs_categories, hue=noms_categories,
plt.title('Répartition des Sentiments parmi les Avis Clients', fontsize=1
plt.xlabel('Type de Sentiment', fontsize=12)
plt.ylabel('Nombre d\'Avis total', fontsize=12)
plt.show()

print('-> La colonne analysée est :', colonne_texte)
```

```
print('-> Total des reviews parcourues :', len(liste_textes))
print('-> Décompte final :', compteur_sentiments)
```




```
-> La colonne analysée est : reviewText
-> Total des reviews parcourues : 20000
-> Décompte final : Counter({'Positif': 13127, 'Neutre': 4324, 'Négatif':
2549})
```

2. convert the text to lowercase, remove punctuation and numbers, filter out common stopwords, and then lemmatize the remaining words using their appropriate part-of-speech tags.

In [20]:
```
# Réglage initial de NLTK
import re
import nltk
from textblob import TextBlob, Word
from nltk.corpus import stopwords
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('averaged_perceptron_tagger_eng', quiet=True)
nltk.download('wordnet', quiet=True)

# Extraction des stopwords classiques anglais
mots_vides = set(stopwords.words('english'))

# Fonction pour aider à identifier le type de mot (verbe, nom...) pour la
def trouver_tag_wordnet(tag_pos):
    if tag_pos.startswith('J'):
        return 'a' # Adjectifs en wordnet
    if tag_pos.startswith('V'):
        return 'v' # Verbes
    if tag_pos.startswith('N'):
        return 'n' # Noms
    if tag_pos.startswith('R'):
```

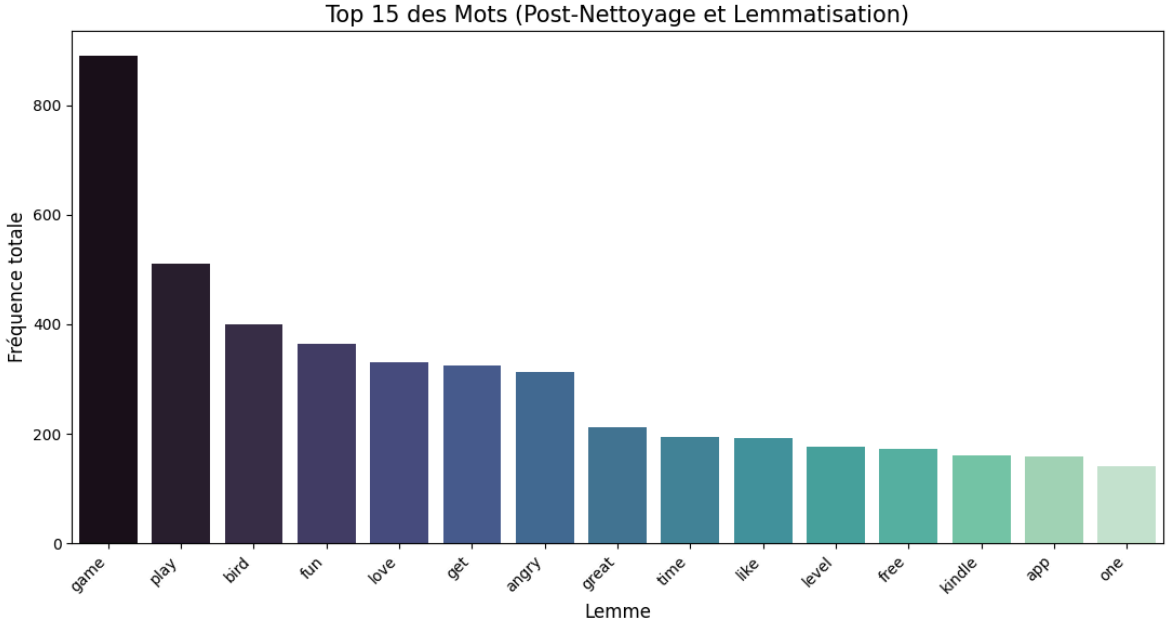
```
        return 'r' # Adverbes
    return 'n' # Nominal par défaut

mots_lemmatises = []
tags_pos_tous = []

# Parcours de chaque texte pour nettoyage et formatage
for texte in liste_textes.tolist()[:1000]: # Optionnel: limitation pour a
    # Nettoyage profond : majuscules, nombres, symboles bizarres
    texte_propre = re.sub(r'[^a-zA-Z\s]', ' ', str(texte).lower())
    texte_propre = re.sub(r'\s+', ' ', texte_propre).strip()

    if texte_propre == '':
        continue

    # Analyse et récupération du type de mot pour chaque élément via Text
    for mot, tag in TextBlob(texte_propre).tags:
        # Exclusion si le mot est trop court ou s'il fait partie des stop
        if mot in mots_vides or len(mot) <= 2:
            continue

        # Lemmatisation
        mot_racine = Word(mot).lemmatize(trouver_tag_wordnet(tag))
        mots_lemmatises.append(mot_racine)
        tags_pos_tous.append(tag)

print('Tokens (mots) après lemmatisation:', len(mots_lemmatises))
```

```
Tokens (mots) après lemmatisation: 14167
```

3. after the text has been fully preprocessed and lemmatized, could you calculate the frequency of each unique word and then display a bar graph showing the top 15 most common words?

In [21]:
```
# 1. Utilisation du module Counter (natif)
decompte_mots = Counter(mots_lemmatises)

# 2. Les 15 mots les plus vus selon Counter
top_quinze = decompte_mots.most_common(15)

if len(top_quinze) > 0:
    # Déballage du tuple (mot, occurrence) via l'étoile *
    labels_mots = [item[0] for item in top_quinze]
    valeur_occurrences = [item[1] for item in top_quinze]

    # 3. Barplot
    plt.figure(figsize=(11, 6))
    sns.barplot(x=labels_mots, y=valeur_occurrences, palette="mako")
    plt.title('Top 15 des Mots (Post-Nettoyage et Lemmatisation)', fontsi
    plt.xlabel('Lemme', fontsize=12)
    plt.ylabel('Fréquence totale', fontsize=12)
    plt.xticks(rotation=45, ha='right', fontsize=10) # Rotation des étiqu

    # On garantit que ce soit centré
    plt.tight_layout()
    plt.show()
else:
    print('Aucun mot pertinent n\'a été traité.')
```

```
/var/folders/m3/b4l6h1m15l500ccxr_h9kx3h0000gn/T/ipykernel_69487/423146829
2.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remove
d in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for
the same effect.

  sns.barplot(x=labels_mots, y=valeur_occurrences, palette="mako")
```



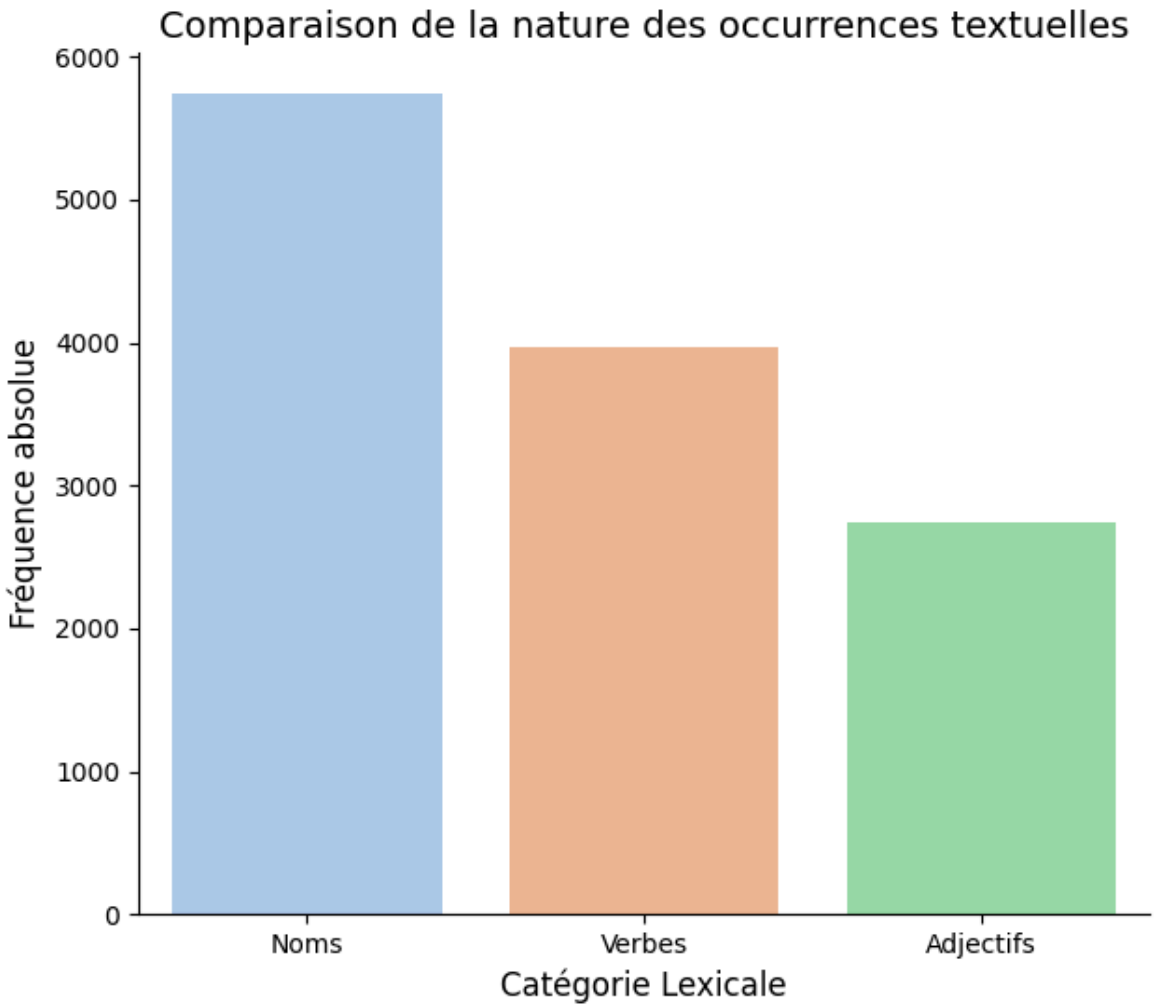
4. generate a word cloud based on the entire corpus of lemmatized review words to provide a visual summary of the most prominent terms.

In [22]:
```
# On joint toutes les listes de lemmes de string avec des espaces
corpus_complet = ' '.join(mots_lemmatises)

# Si le texte n'est pas complètement nul (après les différentes itération
if corpus_complet.strip() != '':

    # Instance de la classe WordCloud
    nuage = WordCloud(
        width=900,
        height=500,
        background_color='ivory',
        colormap='viridis',  # Mieux qu'un jeu standard
        max_words=200       # Pour éviter la surcharge visuelle
    ).generate(corpus_complet)

    # Paramètres d'Axe de la fenêtre Matplotlib
    plt.figure(figsize=(11, 9))
    plt.imshow(nuage, interpolation='bilinear') # Meilleur lissage
    plt.axis('off')                             # Retirer les grilles et
    plt.title('Aperçu Graphique du Ton des Avis (Nuage de Mots)', fontsiz
    plt.show()

else:
    print('Pas assez d\'informations pour construire ce nuage de mots.')
```

Aperçu Graphique du Ton des Avis (Nuage de Mots)

5. Visualize the frequency of nouns, verbs, adjectives in this text.

In [23]:
```
# Initialisation du dictionnaire des catégories grammaticales (Part of Sp
# Basé sur N (Noms), V (Verbes), J (Adjectifs/Adjectives)
repartition_tags = Counter({'Noms': 0, 'Verbes': 0, 'Adjectifs': 0})

for etiquette in tags_pos_tous:
    if etiquette.startswith('N'):
        repartition_tags['Noms'] += 1
    elif etiquette.startswith('V'):
        repartition_tags['Verbes'] += 1
    elif etiquette.startswith('J'):
        repartition_tags['Adjectifs'] += 1

plt.figure(figsize=(7, 6))

labels_postags = ['Noms', 'Verbes', 'Adjectifs']
valeurs_postags = [repartition_tags['Noms'], repartition_tags['Verbes'],

sns.barplot(x=labels_postags, y=valeurs_postags, palette='pastel', hue=la
plt.title('Comparaison de la nature des occurrences textuelles', fontsize
plt.xlabel('Catégorie Lexicale', fontsize=12)
plt.ylabel('Fréquence absolue', fontsize=12)

# Un graphique simple sans bordures épaisses (effet "pureté" à la seaborn
sns.despine()
plt.show()
```


Comparaison de la nature des occurrences textuelles
Fréquence absolue
6000
5000
4000
3000
2000
1000
0
Noms
Verbes
Adjectifs
Catégorie Lexicale